

TNA Minimal Implementation v1

Pseudo-code + Python Skeleton

0. Design Philosophy

This is not a machine learning model or a classical solver.

It is a loop that:

1. **detects failures**
2. **interprets them as boundaries**
3. **generates expansions**
4. **evaluates consistency**
5. **iterates indefinitely**

Core principle:

boundary $\rightarrow$ indeterminacy $\rightarrow$ pressure $\rightarrow$ expansion

1. Minimal Pseudo-code

```
Initialize Domain  $\Omega$ 

while True:

    # 1. Evaluate domain
    results = evaluate( $\Omega$ )

    # 2. Detect boundary
     $\partial\Omega$  = detect_boundary(results)

    # 3. Measure indeterminacy
    I_B = compute_indeterminacy( $\partial\Omega$ )
```

```

# 4. Compute pressure
F_B = aggregate(I_B)

# 5. Expansion trigger
if F_B > threshold:

    candidates = generate_expansions( $\Omega$ )

     $\Omega$  = select_best(candidates)

else:
     $\Omega$  = probe( $\Omega$ )

log( $\Omega$ ,  $\partial\Omega$ , F_B)

```

2. Python Skeleton

Minimal, extensible, and faithful to TNA principles:

```

import random
from typing import List, Dict

# -----
# Domain Representation
# -----

class Domain:
    def __init__(self, parameters: Dict[str, float]):
        self.parameters = parameters

    def mutate(self):
        """Simple expansion: perturb parameters"""
        new_params = {
            k: v + random.uniform(-0.5, 0.5)
            for k, v in self.parameters.items()
        }
        return Domain(new_params)

```

```

# -----
# Evaluation
# -----

def evaluate(domain: Domain) -> List[float]:
    """
    Simulated evaluation function.
    Replace with real system (physics, ML, cognition).
    """
    results = []
    for v in domain.parameters.values():
        # instability when |v| grows
        score = 1.0 / (1.0 + abs(v))
        results.append(score)
    return results

# -----
# Boundary Detection
# -----

def detect_boundary(results: List[float], threshold=0.2) -> List[int]:
    """
    Boundary = low-reliability regions
    """
    return [i for i, r in enumerate(results) if r < threshold]

# -----
# Indeterminacy
# -----

def compute_indeterminacy(boundary: List[int]) -> float:
    """
    Proxy: number of failing regions
    """
    return len(boundary)

# -----
# Pressure

```

```

# -----

def compute_pressure(indeterminacy: float) -> float:
    return indeterminacy

# -----
# Expansion
# -----

def generate_expansions(domain: Domain, n=5):
    return [domain.mutate() for _ in range(n)]

# -----
# Selection
# -----

def select_best(domains):
    """
    Select domain with highest average reliability
    """
    scored = []
    for d in domains:
        results = evaluate(d)
        score = sum(results) / len(results)
        scored.append((score, d))

    scored.sort(key=lambda x: x[0], reverse=True)
    return scored[0][1]

# -----
# Probe (no expansion)
# -----

def probe(domain: Domain):
    """
    Small drift when no pressure
    """
    return domain.mutate()

```

```

# -----
# Main Loop
# -----

def run_tna(initial_domain: Domain, steps=50, pressure_threshold=2):

     $\Omega$  = initial_domain

    for step in range(steps):

        results = evaluate( $\Omega$ )

        boundary = detect_boundary(results)

        I_B = compute_indeterminacy(boundary)

        F_B = compute_pressure(I_B)

        print(f"\nStep {step}")
        print(f"Parameters: { $\Omega$ .parameters}")
        print(f"Boundary size: {len(boundary)} | Pressure: {F_B:.2f}")

        if F_B > pressure_threshold:
            print("→ Expansion triggered")

            candidates = generate_expansions( $\Omega$ )

             $\Omega$  = select_best(candidates)

        else:
             $\Omega$  = probe( $\Omega$ )

    return  $\Omega$ 

# -----
# Example Run
# -----

if __name__ == "__main__":

```

```
initial = Domain({
    "x": 1.0,
    "y": 2.0,
    "z": 3.0
})

final_domain = run_tna(initial, steps=30)

print("\nFinal Domain:", final_domain.parameters)
```

3. What This Implements

This minimal system already encodes:

- $\Omega \rightarrow$ **parameter space**
- $R(x) \rightarrow$ **evaluation function**
- $\partial\Omega \rightarrow$ **low-performance regions**
- $I_B \rightarrow$ **number of failures**
- $F_B \rightarrow$ **pressure**
- **Expansion $\rightarrow$ mutation + selection**

👉 It is a **functional TNA loop**, not just a concept.

4. How to Upgrade

To move toward a real system:

1. Replace `evaluate()`

- physics simulator
 - neural network
 - symbolic system
 - cognitive model
-

2. Improve boundary detection

- gradients
 - divergence detection
 - uncertainty estimation
-

3. Advanced Expansion Engine

- evolutionary search
 - symbolic regression
 - LLM-generated hypotheses
-

4. Add Memory

```
memory = {  
    "domains": [],  
    "scores": [],  
    "boundaries": []  
}
```

5. Integrate Benchmark Suite

- automatically run T1–T5
 - compute TCI (TNA Capability Index)
-

5. Key Property

This system does not learn from data.

It learns from:

■ *its own structural failures*

6. Canonical Insight

What you now have (even in this minimal version) is:

■ *a system that generates theory by confronting its own limits*